

Web Application Security Assessment Report

Prepared By: Mehedi

Target Application: Damn Vulnerable Web Application (DVWA)

Environment: Localhost (XAMPP on Windows)

Date: May 16, 2026

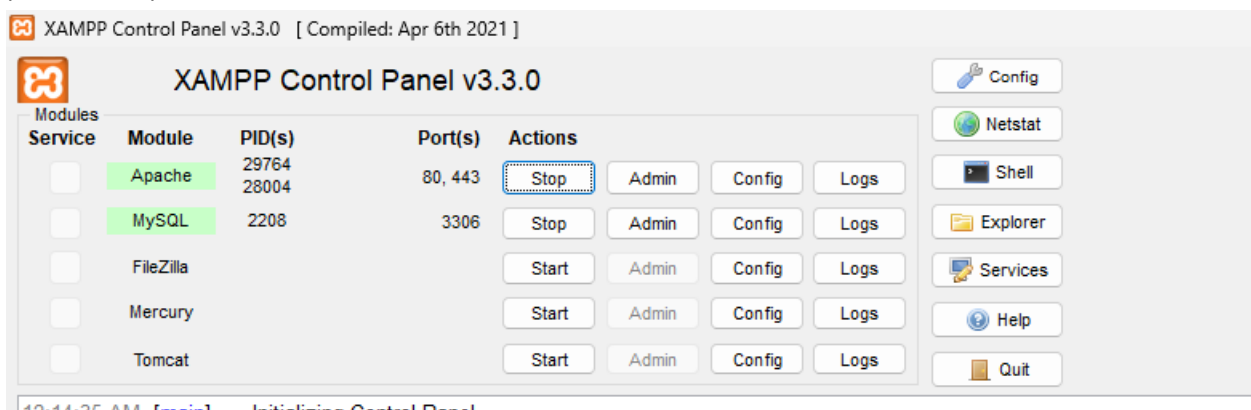
1. Executive Summary

This report documents the setup of a local security laboratory and the subsequent vulnerability assessment of the hosted web application. The environment was built using XAMPP on Windows to host the Damn Vulnerable Web Application (DVWA). The primary objective was to demonstrate and document critical security flaws, specifically focusing on SQL Injection.

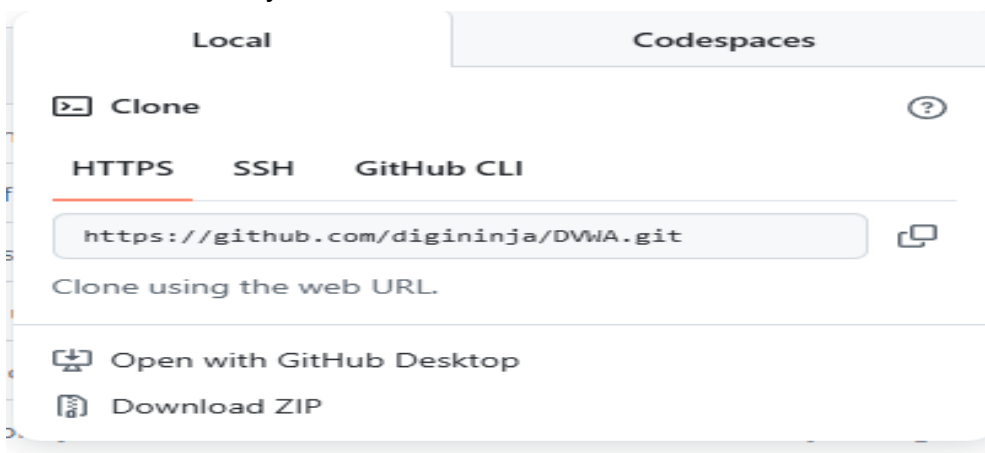
2. Project Phases (Step-by-Step)

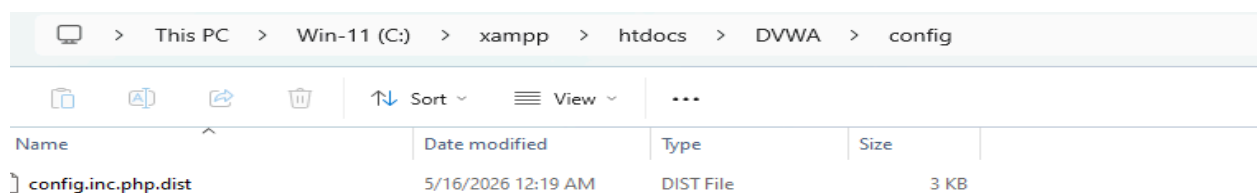
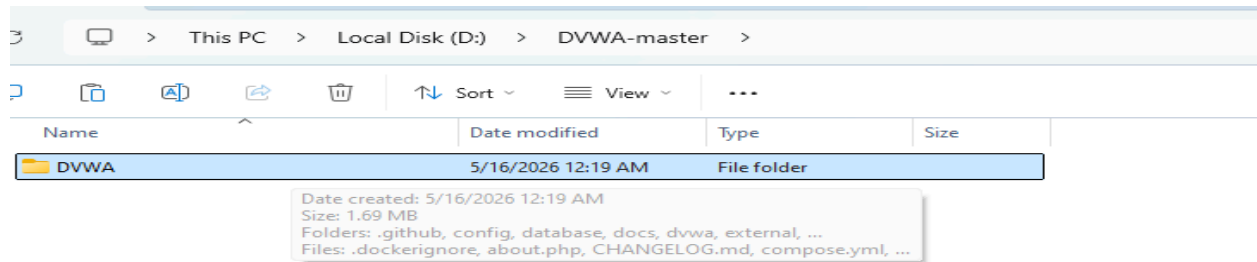
Phase 1: Lab Environment Setup

- XAMPP Installation: Installed XAMPP to manage Apache (Web Server) and MySQL (Database).



- DVWA Deployment: Cloned the DVWA repository from GitHub and moved the files to the htdocs directory.





- Configuration: Renamed config.inc.php.dist to config.inc.php and configured database credentials (user: 'dvwa', password: 'p@ssw0rd').

```
File Edit View

<?php

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$dbms = getenv('DBMS') ?: 'MySQL';
#$dbms = 'PGSQL'; // Currently disabled

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
# See README.md for more information on this.
$DVWA = array();
$DVWA['db_server'] = getenv('DB_SERVER') ?: '127.0.0.1';
$DVWA['db_database'] = getenv('DB_DATABASE') ?: 'dvwa';
$DVWA['db_user'] = getenv('DB_USER') ?: 'dvwa';
$DVWA['db_password'] = getenv('DB_PASSWORD') ?: 'p@ssw0rd';
$DVWA['db_port'] = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
$DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$DVWA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ?: 'impossible';

# Default locale
```

- PHP Tuning: Modified php.ini to set allow_url_include to 'On' for application compatibility.

```
; Whether to allow the treatment of URLs (like http:// or ftp://) as files.  
; https://php.net/allow-url-fopen  
allow_url_fopen=On
```

allow_url_include

```
; Whether to allow include/require to open URLs (like https:// or ftp://) as files.  
; https://php.net/allow-url-include  
allow_url_include=On
```

PHP

PHP version: 8.2.12

PHP function display_errors: Enabled

PHP function display_startup_errors: Enabled

PHP function allow_url_include: Enabled - Feature deprecated

PHP function allow_url_fopen: Enabled

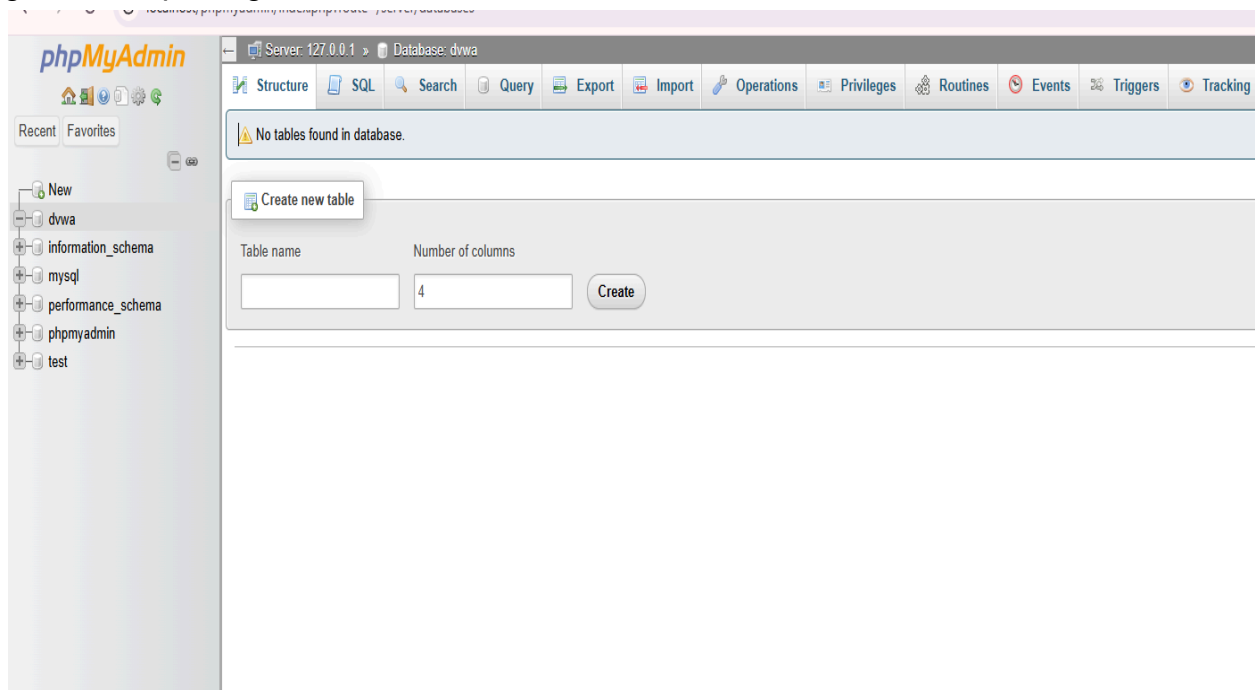
PHP module gd: Installed

PHP module mysql: Installed

PHP module pdo_mysql: Installed

Phase 2: Database Initialization & Troubleshooting

- User Privileges: Created a dedicated database user 'dwva' in phpMyAdmin and granted all privileges to resolve initial connection errors.



phpMyAdmin

Server: 127.0.0.1

Databases SQL Status User accounts Export Import Settings Replication Variables Charsets Engines Plugins

Recent Favorites

New dvwa information_schema mysql performance_schema phpmyadmin test

Add user account

Login Information

User name:

Host name: %

Password: Strength: Weak

Re-type:

Authentication plugin:

Generate password:

Database for user account

☐ Create database with same name and grant all privileges.
☐ Grant all privileges on wildcard name (username_%).
☒ Grant all privileges on database dvwa.

Global privileges ☐ Check all

Note: MySQL privilege names are expressed in English.

Data

☐ SELECT
☐ INSERT
☐ UPDATE
☐ DELETE
☐ FILE

Structure

☐ CREATE
☐ ALTER
☐ INDEX
☐ DROP
☐ CREATE TEMPORARY TABLES
☐ SHOW VIEW
☐ CREATE ROUTINE

Administration

☐ GRANT
☐ SUPER
☐ PROCESS
☐ RELOAD
☐ SHUTDOWN
☐ SHOW DATABASES
☐ LOCK TABLES

Resource limits

Note: Setting these options to 0 (zero) removes the limit.

MAX QUERIES PER HOUR:

MAX UPDATES PER HOUR:

MAX CONNECTIONS PER HOUR:

MAX USER CONNECTIONS:

- Database Creation: Successfully initialized the database tables through the DVWA setup page.

Phase 3: Security Configuration

Accessed the 'DVWA Security' panel and set the security level to 'Low' to facilitate the initial vulnerability testing phase.

3. Security Finding: SQL Injection (Critical)

Vulnerability Name	SQL Injection (In-band/Error-based)
Severity	CRITICAL
Vulnerable Component	User ID Input Field

Description:

The application fails to properly sanitize or filter user-supplied input in the 'User ID' field. This allows an attacker to inject malicious SQL commands that alter the intended query logic on the backend database.

Proof of Concept (PoC):

Payload Used: ' OR '1'='1

By entering this payload into the User ID field, the application returned the full list of users registered in the database, bypassing the intended single-user lookup logic.

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info
About

Logout

Vulnerability: SQL Injection

User ID:

ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob
Surname: Smith

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

4. Impact

- Unauthorized Data Access: Full disclosure of sensitive user information.
- Potential Account Takeover: Access to password hashes for offline cracking.

5. Recommendation

Implement Parameterized Queries (Prepared Statements) to ensure that user input is never interpreted as executable code. Additionally, enforce strict input validation and least-privilege database access.

